

Ejercicios: struct y match
Unidad 0: Herramientas y preliminares

Eduardo Acuña Yeomans

Lenguajes de Programación
Semestre 2026-2

Lic. en Ciencias de la Computación
Universidad de Sonora

Sesión 3

Esta hoja no se entrega y no se califica. Empieza copiando a tu archivo lo que hicimos en clase, que es de donde salen todos los ejercicios:

```
(struct circulo (radio) #:transparent)
(struct rectangulo (base altura) #:transparent)
(struct cuadrado (lado) #:transparent)

(define (area f)
  (match f
    [(circulo r) (* 3.1416 r r)]
    [(rectangulo b h) (* b h)]
    [(cuadrado l) (* l l)]))
```

Definir tipos nuevos

1. Dos figuras más Define los structs de un triángulo, que se describe con base y altura, y de un rombo, que se describe con sus dos diagonales. Ponles `#:transparent`, como los demás.

Usa estos nombres para que las salidas de abajo coincidan: `triangulo` con campos `base` y `altura`, y `rombo` con campos `diagonal-mayor` y `diagonal-menor`.

2. Las tres cosas que te dio struct Comprueba en el REPL que la definición te dio constructor, selectores y predicado:

```
> (define t (triangulo 6 4))
> t
(triangulo 6 4)
> (triangulo-base t)
6
> (triangulo-altura t)
4
> (triangulo? t)
```

```
#t
> (rombo? t)
#f
```

3. Un error que conviene provocar Teclea `(triangulo 6)` y lee el mensaje completo. ¿Qué te está diciendo Racket, y por qué es una buena noticia?

Procesar por variante

4. Extender area Agrega los dos casos nuevos. El área de un triángulo es base por altura entre dos. La de un rombo, el producto de sus diagonales entre dos.

```
(area (triangulo 6 4)) = 12
(area (rombo 8 6))    = 24
(area (circulo 3))    = 28.2744
(area (cuadrado 4))   = 16
```

5. perimetro, desde cero Escribe una función nueva que calcule el perímetro. Con las tres figuras de clase basta: círculo, rectángulo y cuadrado.

```
(perimetro (circulo 3))    = 18.8496
(perimetro (rectangulo 2 5)) = 14
(perimetro (cuadrado 4))   = 16
```

Sugerencia: por qué el perímetro no lleva triángulo ni rombo

Con base y altura no alcanza para saber cuánto miden los lados de un triángulo. La representación que elegimos sirve para una pregunta y no para la otra, y eso no es un defecto tuyo, es una decisión de diseño con consecuencias.

6. El comodín Escribe `tiene-lados-iguales?`, que devuelve `#t` para el cuadrado y el rombo, y `#f` para las demás. Los campos no te interesan: úsalos con `_`.

```
(tiene-lados-iguales? (cuadrado 4)) = #t
(tiene-lados-iguales? (rombo 8 6))  = #t
(tiene-lados-iguales? (rectangulo 2 5)) = #f
```

Sugerencia: un caso que atrapa todo

Un patrón que es solo `_`, sin paréntesis, acepta cualquier cosa. Va al final: si lo pones primero, se traga todos los demás casos.

7. Sin escribir código Define un `struct` nuevo (por ejemplo (`struct pentagono (lado #:transparent)`) y llama a `area` con él. Predice el error *antes* de teclearlo, y después compruébalo. ¿En qué momento aparece: al escribir el archivo, al presionar Run, o al hacer la llamada?

Soluciones

Clave de Respuesta: definir tipos nuevos

```
(struct triangulo (base altura) #:transparent)
(struct rombo (diagonal-mayor diagonal-menor) #:transparent)
```

Ejercicio 3. `(triangulo 6)` responde:

```
triangulo: arity mismatch;
the expected number of arguments does not match the given number
  expected: 2
   given: 1
```

Un triángulo tiene exactamente dos campos y Racket lo sabe, así que te avisa en el momento en lugar de dejarte una figura a medio construir que reviente mucho después y en otro lado. Con la lista de la sesión pasada, `(list 'triangulo 6)` habría pasado sin protestar.

Clave de Respuesta: procesar por variante

```
(define (area f)
  (match f
    [(circulo r) (* 3.1416 r r)]
    [(rectangulo b h) (* b h)]
    [(cuadrado l) (* l l)]
    [(triangulo b h) (/ (* b h) 2)]
    [(rombo D d) (/ (* D d) 2)]))

(define (perimetro f)
  (match f
    [(circulo r) (* 2 3.1416 r)]
    [(rectangulo b h) (* 2 (+ b h))]
    [(cuadrado l) (* 4 l)]))

(define (tiene-lados-iguales? f)
  (match f
    [(cuadrado _) #t]
    [(rombo _ _) #t]
    [_ #f]))
```

En `tiene-lados-iguales?` hay dos usos distintos del comodín: dentro de un patrón (`(rombo _ _)`, que sí exige que sea un rombo pero ignora sus dos campos) y como patrón completo (`_`, que acepta lo que sea).

Ejercicio 7. El error es `match: no matching clause for (pentagono 5)`, y aparece **al hacer la llamada**. Ni al escribir el archivo ni al presionar Run: nadie revisa que tu `match` cubra todas las variantes que existen. Por eso “un caso por variante” no es cuestión de estilo.